

```
In [8]: nyc.head()
Out[8]:
```

	Date	Value	Anomaly
0	189501	34.2	-3.2
1	189601	34.7	-2.7
2	189701	35.5	-1.9
3	189801	39.6	2.2
4	189901	36.4	-1.0

```
In [9]: nyc.tail()
Out[9]:
```

	Date	Value	Anomaly
119	201401	35.5	-1.9
120	201501	36.1	-1.3
121	201601	40.8	3.4
122	201701	42.8	5.4
123	201801	38.7	1.3

Cleaning the Data

We'll soon use Seaborn to graph the Date-Value pairs and a regression line. When plotting data from a DataFrame, Seaborn labels a graph's axes using the DataFrame's column names. For readability, let's rename the 'Value' column as 'Temperature':

[lick here to view code image](#)

```
In [10]: nyc.columns = ['Date', 'Temperature', 'Anomaly']

In [11]: nyc.head(3)
Out[11]:
```

	Date	Temperature	Anomaly
0	189501	34.2	-3.2
1	189601	34.7	-2.7
2	189701	35.5	-1.9

Seaborn labels the tick marks on the x-axis with Date values. Since this example processes only January temperatures, the x-axis labels will be more readable if they do not contain 01 (for January), we'll remove it from each Date. First, let's check the column's type:

```
In [12]: nyc.Date.dtype
Out[12]: dtype('int64')
```

The values are integers, so we can divide by 100 to truncate the last two digits. Recall that each column in a DataFrame is a Series. Calling Series method floordiv performs *integer division* on every element of the Series:

[lick here to view code image](#)

```
In [13]: nyc.Date = nyc.Date.floordiv(100)

In [14]: nyc.head(3)
```

```
Out[14]:
   Date  Temperature  Anomaly
0  1895           34.2    -3.2
1  1896           34.7    -2.7
2  1897           35.5    -1.9
```

Calculating Basic Descriptive Statistics for the Dataset

For some quick statistics on the dataset's temperatures, call `describe` on the `Temperature` column. We can see that there are 124 observations, the mean value of the observations is 37.60, and the lowest and highest observations are 26.10 and 47.60 degrees, respectively:

[lick here to view code image](#)

```
In [15]: pd.set_option('precision', 2)

In [16]: nyc.Temperature.describe()
Out[16]:
count      124.00
mean        37.60
std         4.54
min         26.10
25%         34.58
50%         37.60
75%         40.60
max         47.60
Name: Temperature, dtype: float64
```

Forecasting Future January Average High Temperatures

The **SciPy (Scientific Python) library** is widely used for engineering, science and math in Python. Its **stats module** provides function **linregress**, which calculates a regression line's *slope* and *intercept* for a given set of data points:

[lick here to view code image](#)

```
In [17]: from scipy import stats

In [18]: linear_regression = stats.linregress(x=nyc.Date,
...:                                         y=nyc.Temperature)
...:
```

Function `linregress` receives two one-dimensional arrays ² of the same length representing the data points' *x*- and *y*-coordinates. The keyword arguments `x` and `y` represent the independent and dependent variables, respectively. The object returned by `linregress` contains the regression line's `slope` and `intercept`:

²These arguments also can be one-dimensional array-like objects, such as lists or pandas `Series`.

[lick here to view code image](#)

```
In [19]: linear_regression.slope
Out[19]: 0.00014771361132966167

In [20]: linear_regression.intercept
Out[20]: 8.694845520062952
```

We can use these values with the simple linear regression equation for a straight line, $y = mx + b$, to predict the average January temperature in New York City for a given year. Let's predict the average Fahrenheit temperature for January of 2019. In the following calculation, `linear_regression.slope` is m , 2019 is x (the date value for which you'd like to predict the temperature), and `linear_regression.intercept` is b :

[lick here to view code image](#)

```
In [21]: linear_regression.slope * 2019 + linear_regression.intercept
Out[21]: 38.51837136113298
```

We also can approximate what the average temperature might have been in the years before 1895. For example, let's approximate the average temperature for January of 1890:

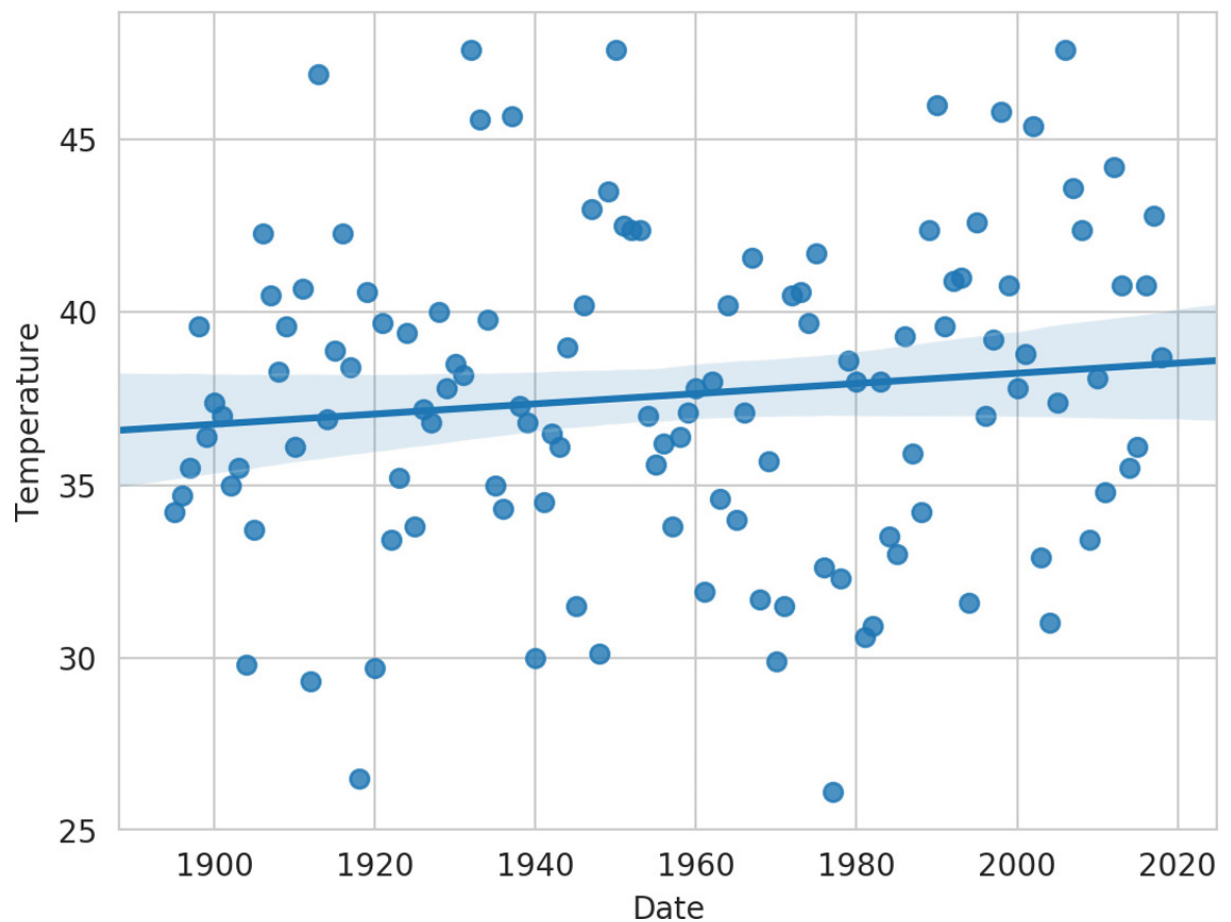
[lick here to view code image](#)

```
In [22]: linear_regression.slope * 1890 + linear_regression.intercept
Out[22]: 36.612865774980335
```

For this example, we had data for 1895–2018. You should expect that the further you go outside this range, the less reliable the predictions will be.

Plotting the Average High Temperatures and a Regression Line

Next, let's use Seaborn's **regplot function** to plot each data point with the dates on the x -axis and the temperatures on the y -axis. The `regplot` function creates the **scatter plot** or **scattergram** below in which the scattered dots represent the Temperatures for the given Dates, and the straight line displayed through the points is the regression line:



First, close the prior Matplotlib window if you have not done so already—otherwise, `regplot` will use the existing window that already contains a graph. Function `regplot`'s `x` and `y` keyword arguments are one-dimensional arrays ³ of the same length representing the *x-y* coordinate pairs to plot. Recall that pandas automatically creates attributes for each column name if the name can be a valid Python identifier: ⁴

³These arguments also can be one-dimensional array-like objects, such as lists or pandas Series.

⁴For readers with a more statistics background, the shaded area surrounding the regression line is the 95% *confidence interval* for the regression line (https://en.wikipedia.org/wiki/Simple_linear_regression#Confidence_interval) to draw the diagram without a confidence interval, add the keyword argument `ci=None` to the `regplot` functions argument list.

[lick here to view code image](#)

```
In [23]: import seaborn as sns

In [24]: sns.set_style('whitegrid')

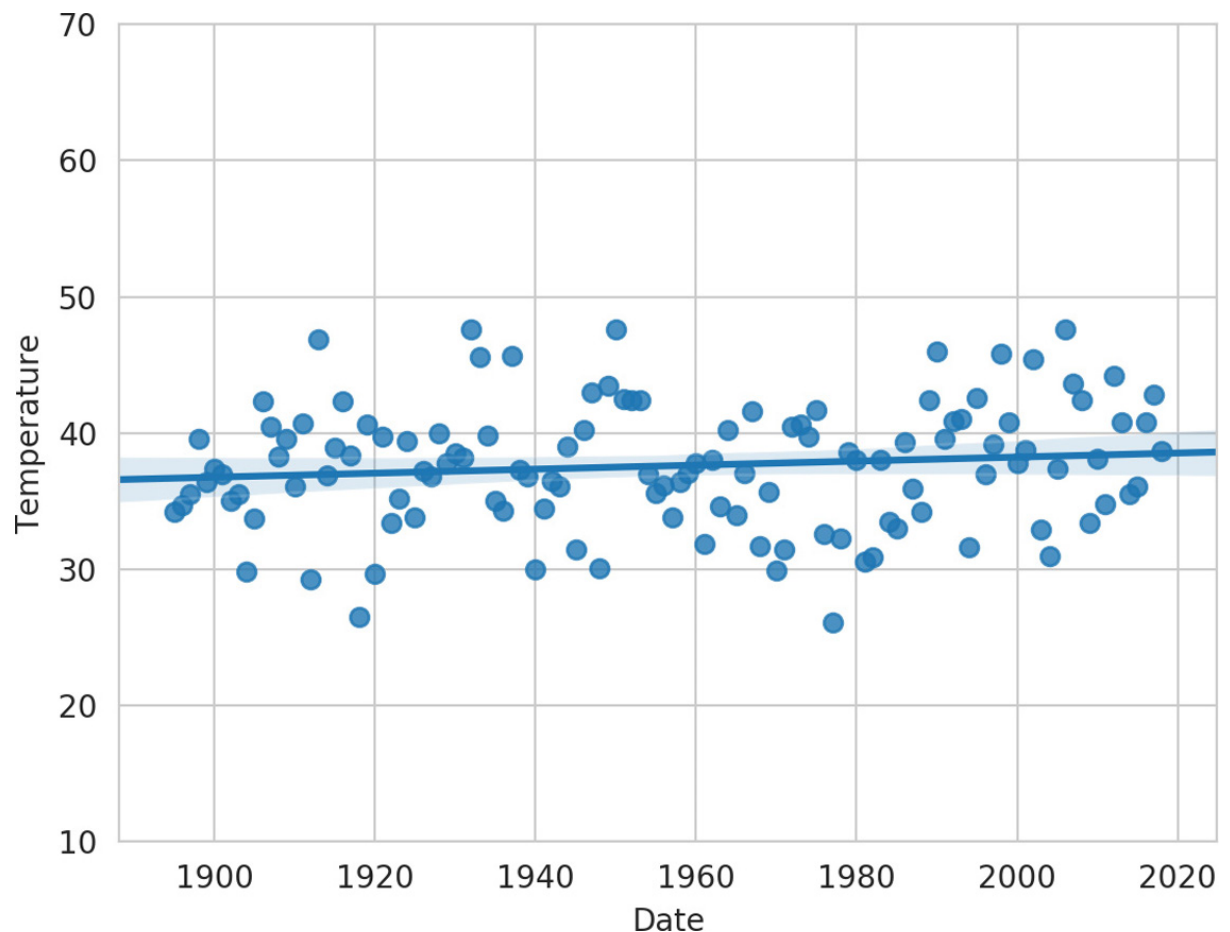
In [25]: axes = sns.regplot(x=nyc.Date, y=nyc.Temperature)
```

The regression line's slope (lower at the left and higher at the right) indicates a warming trend over the last 124 years. In this graph, the *y*-axis represents a 21.5-degree temperature range between the minimum of 26.1 and the maximum of 47.6, so the data appears to be

pread significantly above and below the regression line, making it difficult to see the linear relationship. This is a common issue in data analytics visualizations. When you have axes that reflect different kinds of data (dates and temperatures in this case), how do you reasonably determine their respective scales? In the preceding graph, this is purely an issue of the graph's height—Seaborn and Matplotlib *auto-scale* the axes, based on the data's range of values. We can scale the *y*-axis range of values to emphasize the linear relationship. Here, we scaled the *y*-axis from a 21.5-degree range to a 60-degree range (from 10 to 70 degrees):

[lick here to view code image](#)

```
In [26]: axes.set_ylim(10, 70)
Out[26]: (10, 70)
```



Getting Time Series Datasets

Here are some popular sites where you can find time series to use in your studies:

Sources time-series dataset

<https://data.gov/>

This is the U.S. government's open data portal. Searching for "time series" yields over 7200 time-series datasets.

```
https://www.ncdc.noaa.gov/cag/
```

The National Oceanic and Atmospheric Administration (NOAA) Climate at a Glance portal provides both global and U.S. weather-related time series.

```
https://www.esrl.noaa.gov/psd/data/timeseries/
```

NOAA's Earth System Research Laboratory (ESRL) portal provides monthly and seasonal climate-related time series.

```
https://www.quandl.com/search
```

Quandl provides hundreds of free financial-related time series, as well as fee-based time series.

```
https://datamarket.com/data/list/?q=provider:tsdl
```

The Time Series Data Library (TSDL) provides links to hundreds of time series datasets across many industries.

```
http://archive.ics.uci.edu/ml/datasets.html
```

The University of California Irvine (UCI) Machine Learning Repository contains dozens of time-series datasets for a variety of topics.

```
http://inforumweb.umd.edu/econdata/econdata.html
```

The University of Maryland's EconData service provides links to thousands of economic time series from various U.S. government agencies.

10.17 WRAP-UP

In this chapter, we discussed the details of crafting valuable classes. You saw how to define a class, create objects of the class, access an object's attributes and call its methods. You defined the special method `__init__` to create and initialize a new object's data attributes.

We discussed controlling access to attributes and using properties. We showed that all object

Attributes may be accessed directly by a client. We discussed identifiers with single leading underscores (`_`), which indicate attributes that are not meant to be accessed by client code. We showed how to implement “private” attributes via the double-leading-underscore (`__`) naming convention, which tells Python to mangle an attribute’s name.

We implemented a card shuffling and dealing simulation consisting of a `Card` class and a `DeckOfCards` class that maintained a list of `Cards`, and displayed the deck both as strings and as card images using `Matplotlib`. We introduced special methods `__repr__`, `__str__` and `__format__` for creating string representations of objects.

Next, we looked at Python’s capabilities for creating base classes and subclasses. We showed how to create a subclass that inherits many of its capabilities from its superclass, then adds more capabilities, possibly by overriding the base class’s methods. We created a list containing both base class and subclass objects to demonstrate Python’s polymorphic programming capabilities.

We introduced operator overloading for defining how Python’s built-in operators work with objects of custom class types. You saw that overloaded operator methods are implemented by overriding various special methods that all classes inherit from class `object`. We discussed the Python exception class hierarchy and creating custom exception classes.

We showed how to create a named tuple that enables you to access tuple elements via attribute names rather than index numbers. Next, we introduced Python 3.7’s new data classes, which can autogenerate various boilerplate code commonly provided in class definitions, such as the `__init__`, `__repr__` and `__eq__` special methods.

You saw how to write unit tests for your code in docstrings, then execute those tests conveniently via the `doctest` module’s `testmod` function. Finally, we discussed the various namespaces that Python uses to determine the scopes of identifiers.

In the next part of the book, we present a series of implementation case studies that use a mix of AI and big-data technologies. We explore natural language processing, data mining Twitter, IBM Watson and cognitive computing, supervised and unsupervised machine learning, and deep learning with convolutional neural networks and recurrent neural networks. We discuss big-data software and hardware infrastructure, including NoSQL databases, Hadoop and Spark with a major emphasis on performance. You’re about to see some really cool stuff!

11. Natural Language Processing (NLP)

Objectives

In this chapter you'll:

- Perform natural language processing (NLP) tasks, which are fundamental to many of the forthcoming data science case study chapters.
- Run lots of NLP demos.
- Use the TextBlob, NLTK, Textstatistic and spaCy NLP libraries and their pretrained models to perform various NLP tasks.
- Tokenize text into words and sentences.
- Use parts-of-speech tagging.
- Use sentiment analysis to determine whether text is positive, negative or neutral.
- Detect the language of text and translate between languages using TextBlob's Google Translate support.
- Get word roots via stemming and lemmatization.
- Use TextBlob's spell checking and correction capabilities.
- Get word definitions, synonyms and antonyms.
- Remove stop words from text.
- Create word clouds.
- Determine text readability with Textstatistic.
- Use the spaCy library for named entity recognition and similarity detection.

Outline

1.1 Introduction

1.2 TextBlob

1.2.1 Create a TextBlob

1.2.2 Tokenizing Text into Sentences and Words

[1.2.3 Parts-of-Speech Tagging](#)

[1.2.4 Extracting Noun Phrases](#)

[1.2.5 Sentiment Analysis with TextBlob's Default Sentiment Analyzer](#)

[1.2.6 Sentiment Analysis with the NaiveBayesAnalyzer](#)

[1.2.7 Language Detection and Translation](#)

[1.2.8 Inflection: Pluralization and Singularization](#)

[1.2.9 Spell Checking and Correction](#)

[1.2.10 Normalization: Stemming and Lemmatization](#)

[1.2.11 Word Frequencies](#)

[1.2.12 Getting Definitions, Synonyms and Antonyms from WordNet](#)

[1.2.13 Deleting Stop Words](#)

[1.2.14 n-grams](#)

[1.3 Visualizing Word Frequencies with Bar Charts and Word Clouds](#)

[1.3.1 Visualizing Word Frequencies with Pandas](#)

[1.3.2 Visualizing Word Frequencies with Word Clouds](#)

[1.4 Readability Assessment with Textstatistic](#)

[1.5 Named Entity Recognition with spaCy](#)

[1.6 Similarity Detection with spaCy](#)

[1.7 Other NLP Libraries and Tools](#)

[1.8 Machine Learning and Deep Learning Natural Language Applications](#)

[1.9 Natural Language Datasets](#)

[1.10 Wrap-Up](#)

11.1 INTRODUCTION

Your alarm wakes you, and you hit the “Alarm Off” button. You reach for your smartphone and read your text messages and check the latest news clips. You listen to TV hosts interviewing celebrities. You speak to family, friends and colleagues and listen to their responses. You have a hearing-impaired friend with whom you communicate via sign language and who enjoys close-captioned video programs. You have a blind colleague who reads braille, listens to books being read by a computerized book reader and listens to a screen reader speak about what’s on his computer screen. You read emails, distinguishing

unk from important communications and send email. You read novels or works of non-fiction. You drive, observing road signs like “Stop,” “Speed Limit 35” and “Road Under Construction.” You give your car verbal commands, like “call home,” “play classical music” or ask questions like, “Where’s the nearest gas station?” You teach a child how to speak and read. You send a sympathy card to a friend. You read books. You read newspapers and magazines. You take notes during a class or meeting. You learn a foreign language to prepare for travel abroad. You receive a client email in Spanish and run it through a free translation program. You respond in English knowing that your client can easily translate your email back to Spanish. You are uncertain about the language of an email, but language detection software instantly figures that out for you and translates the email to English.

These are examples of **natural language** communications in text, voice, video, sign language, braille and other forms with languages like English, Spanish, French, Russian, Chinese, Japanese and hundreds more. In this chapter, you’ll master many natural language processing (NLP) capabilities through a series of hands-on demos and IPython sessions. You’ll use many of these NLP capabilities in the upcoming data science case study chapters.

Natural language processing is performed on text collections, composed of Tweets, Facebook posts, conversations, movie reviews, Shakespeare’s plays, historic documents, news items, meeting logs, and so much more. A text collection is known as a **corpus**, the plural of which is **corpora**.

Natural language lacks mathematical precision. Nuances of meaning make natural language understanding difficult. A text’s meaning can be influenced by its context and the reader’s “world view.” Search engines, for example, can get to “know you” through your prior searches. The upside is better search results. The downside could be invasion of privacy.

11.2 TEXTBLOB¹

¹ <https://textblob.readthedocs.io/en/latest/>.

TextBlob is an object-oriented NLP text-processing library that is built on the **NLTK** and **pattern** NLP libraries and simplifies many of their capabilities. Some of the NLP tasks TextBlob can perform include:

- **Tokenization**—splitting text into pieces called **tokens**, which are meaningful units, such as words and numbers.
- **Parts-of-speech (POS) tagging**—identifying each word’s part of speech, such as noun, verb, adjective, etc.
- **Noun phrase extraction**—locating groups of words that represent nouns, such as “red brick factory.”²

² The phrase red brick factory illustrates why natural language is such a difficult subject. Is a red brick factory a factory that makes red bricks? Is it a red factory that makes bricks of any color? Is it a factory built of red bricks that makes products of any type? In today’s music world, it could even be the name of a rock band or the name of a game on your smartphone.

- **Sentiment analysis**—determining whether text has positive, neutral or negative sentiment.
- **Inter-language translation** and **language detection** powered by Google Translate.
- **Inflection** ³ —pluralizing and singularizing words. There are other aspects of inflection that are not part of TextBlob.

³ <https://en.wikipedia.org/wiki/Inflection>.

- **Spell checking** and **spelling correction**.
- **Stemming**—reducing words to their stems by removing prefixes or suffixes. For example, the stem of “varieties” is “variety.”
- **Lemmatization**—like stemming, but produces real words based on the original words’ context. For example, the lemmatized form of “varieties” is “variety.”
- **Word frequencies**—determining how often each word appears in a corpus.
- **WordNet integration** for finding word definitions, synonyms and antonyms.
- **Stop word elimination**—removing common words, such as a, an, the, I, we, you and more to analyze the important words in a corpus.
- **n-grams**—producing sets of consecutive words in a corpus for use in identifying words that frequently appear adjacent to one another.

Many of these capabilities are used as part of more complex NLP tasks. In this section, we’ll perform these NLP tasks using TextBlob and NLTK.

Installing the TextBlob Module

To install TextBlob, open your Anaconda Prompt (Windows), Terminal (macOS/Linux) or shell (Linux), then execute the following command:

[click here to view code image](#)

```
conda install -c conda-forge textblob
```

Windows users might need to run the Anaconda Prompt as an Administrator for proper software installation privileges. To do so, right-click Anaconda Prompt in the start menu and select **More > Run as administrator**.

Once installation completes, execute the following command to download the NLTK corpora used by TextBlob:

[click here to view code image](#)

```
ipython -m textblob.download_corpora
```

These include:

- The Brown Corpus (created at Brown University ⁴) for parts-of-speech tagging.
`https://en.wikipedia.org/wiki/Brown\_Corpus`
- Punkt for English sentence tokenization.
- WordNet for word definitions, synonyms and antonyms.
- Averaged Perceptron Tagger for parts-of-speech tagging.
- conll2000 for breaking text into components, like nouns, verbs, noun phrases and more—known as **chunking** the text. The name conll2000 is from the conference that created the chunking data—Conference on Computational Natural Language Learning.
- Movie Reviews for sentiment analysis.

Project Gutenberg

A great source of text for analysis is the free e-books at Project Gutenberg:

`https://www.gutenberg.org`

The site contains over 57,000 e-books in various formats, including plain text files. These are out of copyright in the United States. For information about Project Gutenberg's Terms of Use and copyright in other countries, see:

`https://www.gutenberg.org/wiki/Gutenberg:Terms\_of\_Use`

In some of this section's examples, we use the plain-text e-book file for Shakespeare's *Romeo and Juliet*, which you can find at:

`https://www.gutenberg.org/ebooks/1513`

Project Gutenberg does not allow programmatic access to its e-books. You're required to copy the books for that purpose. ⁵ To download *Romeo and Juliet* as a plain-text e-book, right click the **Plain Text UTF-8** link on the book's web page, then select **Save Link As...** (Chrome/FireFox), **Download Linked File As...** (Safari) or **Save target as** (Microsoft Edge) option to save the book to your system. Save it as `RomeoAndJuliet.txt` in the `ch11` examples folder to ensure that our code examples will work correctly. For analysis purposes, we removed the Project Gutenberg text before "THE TRAGEDY OF ROMEO AND JULIET", as well as the Project Gutenberg information at the end of the file starting with:

⁵

`https://www.gutenberg.org/wiki/Gutenberg:Information\_About\_Robot\_Access\_to\_ou`



End of the Project Gutenberg EBook of Romeo and Juliet, by William Shakespeare

11.2.1 Create a TextBlob

TextBlob is the fundamental class for NLP with the **textblob module**. Let's create a TextBlob containing two sentences:

6

http://textblob.readthedocs.io/en/latest/api_reference.html#textblob.blob.TextBlob

[click here to view code image](#)

```
In [1]: from textblob import TextBlob

In [2]: text = 'Today is a beautiful day. Tomorrow looks like bad weather.'

In [3]: blob = TextBlob(text)

In [4]: blob
Out[4]: TextBlob("Today is a beautiful day. Tomorrow looks like bad weather.")
```

TextBlobs—and, as you'll see shortly, Sentences and Words—support string methods and can be compared with strings. They also provide methods for various NLP tasks. Sentences, Words and TextBlobs inherit from **BaseBlob**, so they have many common methods and properties.

11.2.2 Tokenizing Text into Sentences and Words

Natural language processing often requires tokenizing text before performing other NLP tasks. TextBlob provides convenient properties for accessing the sentences and words in TextBlobs. Let's use the **sentence property** to get a list of **Sentence** objects:

[click here to view code image](#)

```
In [5]: blob.sentences
Out[5]:
[Sentence("Today is a beautiful day."),
 Sentence("Tomorrow looks like bad weather.")]
```

The **words property** returns a **WordList** object containing a list of **Word** objects, representing each word in the TextBlob with the punctuation removed:

[click here to view code image](#)

```
In [6]: blob.words
Out[6]: WordList(['Today', 'is', 'a', 'beautiful', 'day', 'Tomorrow', 'looks', 'like', 'bad', 'weather'])
```

1.2.3 Parts-of-Speech Tagging

Parts-of-speech (POS) tagging is the process of evaluating words based on their context to determine each word's part of speech. There are eight primary English parts of speech—nouns, pronouns, verbs, adjectives, adverbs, prepositions, conjunctions and interjections (words that express emotion and that are typically followed by punctuation, like “Yes!” or “Ha!”). Within each category there are many subcategories.

Some words have multiple meanings. For example, the words “set” and “run” have hundreds of meanings each! If you look at the [dictionary.com](https://www.dictionary.com) definitions of the word “run,” you’ll see that it can be a verb, a noun, an adjective or a part of a verb phrase. An important use of POS tagging is determining a word’s meaning among its possibly many meanings. This is important for helping computers “understand” natural language.

The **tags property** returns a list of tuples, each containing a word and a string representing its part-of-speech tag:

[lick here to view code image](#)

```
In [7]: blob
Out[7]: TextBlob("Today is a beautiful day. Tomorrow looks like bad weather.")

n [8]: blob.tags
Out[8]:
[('Today', 'NN'),
 ('is', 'VBZ'),
 ('a', 'DT'),
 ('beautiful', 'JJ'),
 ('day', 'NN'),
 ('Tomorrow', 'NNP'),
 ('looks', 'VBZ'),
 ('like', 'IN'),
 ('bad', 'JJ'),
 ('weather', 'NN')]
```

By default, `TextBlob` uses a `PatternTagger` to determine parts-of-speech. This class uses the parts-of-speech tagging capabilities of the *pattern library*:

<https://www.clips.uantwerpen.be/pattern>

You can view the library’s 63 parts-of-speech tags at

<https://www.clips.uantwerpen.be/pages/MBSP-tags>

In the preceding snippet’s output:

- Today, day and weather are tagged as NN—a singular noun or mass noun.
- is and looks are tagged as VBZ—a third person singular present verb.
- a is tagged as DT—a determiner.⁷

⁷ <https://en.wikipedia.org/wiki/Determiner>.

- beautiful and bad are tagged as JJ—an adjective.
- Tomorrow is tagged as NNP—a proper singular noun.
- like is tagged as IN—a subordinating conjunction or preposition.

11.2.4 Extracting Noun Phrases

Let's say you're preparing to purchase a water ski so you're researching them online. You might search for "best water ski." In this case, "water ski" is a noun phrase. If the search engine does not parse the noun phrase properly, you probably will not get the best search results. Go online and try searching for "best water," "best ski" and "best water ski" and see what you get.

A `TextBlob`'s **`noun_phrases`** property returns a `WordList` object containing a list of `Word` objects—one for each noun phrase in the text:

[lick here to view code image](#)

```
In [9]: blob
Out[9]: TextBlob("Today is a beautiful day. Tomorrow looks like bad weather.")

n [10]: blob.noun_phrases
Out[10]: WordList(['beautiful day', 'tomorrow', 'bad weather'])
```

Note that a `Word` representing a noun phrase can contain multiple words. A `WordList` is an extension of Python's built-in list type. `WordLists` provide additional methods for stemming, lemmatizing, singularizing and pluralizing.

11.2.5 Sentiment Analysis with `TextBlob`'s Default Sentiment Analyzer

One of the most common and valuable NLP tasks is **sentiment analysis**, which determines whether text is positive, neutral or negative. For instance, companies might use this to determine whether people are speaking positively or negatively online about their products. Consider the positive word "good" and the negative word "bad." Just because a sentence contains "good" or "bad" does not mean the sentence's sentiment necessarily is positive or negative. For example, the sentence

The food is not good.

clearly has negative sentiment. Similarly, the sentence

The movie was not bad.

clearly has positive sentiment, though perhaps not as positive as something like

The movie was excellent!

Sentiment analysis is a complex machine-learning problem. However, libraries like `TextBlob` have pretrained machine learning models for performing sentiment analysis.

Getting the Sentiment of a `TextBlob`

A `TextBlob`'s **`sentiment`** property returns a **`Sentiment`** object indicating whether the text is positive or negative and whether it's objective or subjective:

[lick here to view code image](#)

```
n [11]: blob
Out[11]: TextBlob("Today is a beautiful day. Tomorrow looks like bad weather.")
```

```
In [12]: blob.sentiment
Out[12]: Sentiment(polarity=0.075000000000000007,
                 subjectivity=0.8333333333333333)
```

In the preceding output, the `polarity` indicates sentiment with a value from -1.0 (negative) to 1.0 (positive) with 0.0 being neutral. The `subjectivity` is a value from 0.0 (objective) to 1.0 (subjective). Based on the values for our `TextBlob`, the overall sentiment is close to neutral, and the text is mostly subjective.

Getting the `polarity` and `subjectivity` from the `Sentiment` Object

The values displayed above probably provide more precision than you need in most cases. This can detract from numeric output's readability. The IPython magic `%precision` allows you to specify the default precision for *standalone* float objects and float objects in *built-in types* like lists, dictionaries and tuples. Let's use the magic to *round* the `polarity` and `subjectivity` values to three digits to the right of the decimal point:

[lick here to view code image](#)

```
In [13]: %precision 3
Out[13]: ' %.3f'

In [14]: blob.sentiment.polarity
Out[14]: 0.075

In [15]: blob.sentiment.subjectivity
Out[15]: 0.833
```

Getting the Sentiment of a Sentence

You also can get the sentiment at the individual sentence level. Let's use the `sentence` property to get a list of `Sentence` objects, then iterate through them and display each `Sentence`'s `sentiment` property:

8

[http://textblob.readthedocs.io/en/latest/api_reference.html#textblob.blob.Sen](http://textblob.readthedocs.io/en/latest/api_reference.html#textblob.blob.Sentence)

[lick here to view code image](#)

```
In [16]: for sentence in blob.sentences:
...:     print(sentence.sentiment)
...:
Sentiment(polarity=0.85, subjectivity=1.0)
Sentiment(polarity=-0.6999999999999999, subjectivity=0.6666666666666666)
```

This might explain why the entire `TextBlob`'s sentiment is close to 0.0 (neutral)—one sentence is positive (0.85) and the other negative (-0.6999999999999999).

11.2.6 Sentiment Analysis with the `NaiveBayesAnalyzer`

By default, a `TextBlob` and the `Sentences` and `Words` you get from it determine sentiment

using a `PatternAnalyzer`, which uses the same sentiment analysis techniques as in the `Pattern` library. The `TextBlob` library also comes with a **`NaiveBayesAnalyzer`**⁹ (module `textblob.sentiments`), which was trained on a database of movie reviews. Naive Bayes⁰ is a commonly used machine learning text-classification algorithm. The following uses the `analyzer` keyword argument to specify a `TextBlob`'s sentiment analyzer. Recall from earlier in this ongoing IPython session that `text` contains 'Today is a beautiful day. Tomorrow looks like bad weather.':

9

https://textblob.readthedocs.io/en/latest/api_reference.html#module-textblob-.en.sentiments.

⁰ https://en.wikipedia.org/wiki/Naive_Bayes_classifier.

[lick here to view code image](#)

```
In [17]: from textblob.sentiments import NaiveBayesAnalyzer

In [18]: blob = TextBlob(text, analyzer=NaiveBayesAnalyzer())

In [19]: blob
Out[19]: TextBlob("Today is a beautiful day. Tomorrow looks like bad weather.")
```

et's use the `TextBlob`'s `sentiment` property to display the text's sentiment using the `NaiveBayesAnalyzer`:

[lick here to view code image](#)

```
In [20]: blob.sentiment
Out[20]: Sentiment(classification='neg', p_pos=0.47662917962091056, p_neg=0.5)
```

n this case, the overall sentiment is classified as negative (`classification='neg'`). The `Sentiment` object's `p_pos` indicates that the `TextBlob` is 47.66% positive, and its `p_neg` indicates that the `TextBlob` is 52.34% negative. Since the overall sentiment is just slightly more negative we'd probably view this `TextBlob`'s sentiment as neutral overall.

Now, let's get the sentiment of each `Sentence`:

[lick here to view code image](#)

```
In [21]: for sentence in blob.sentences:
...:     print(sentence.sentiment)
...:
Sentiment(classification='pos', p_pos=0.8117563121751951, p_neg=0.18824368782
Sentiment(classification='neg', p_pos=0.174363226578349, p_neg=0.825636773421)
```

notice that rather than polarity and subjectivity, the `Sentiment` objects we get from the `NaiveBayesAnalyzer` contain a *classification*—'pos' (positive) or 'neg' (negative)—and `p_pos` (percentage positive) and `p_neg` (percentage negative) values from 0.0 to 1.0.

Once again, we see that the first sentence is positive and the second is negative.

11.2.7 Language Detection and Translation

Inter-language translation is a challenging problem in natural language processing and artificial intelligence. With advances in machine learning, artificial intelligence and natural language processing, services like Google Translate (100+ languages) and Microsoft Bing Translator (60+ languages) can translate between languages instantly.

Inter-language translation also is great for people traveling to foreign countries. They can use translation apps to translate menus, road signs and more. There are even efforts at live speech translation so that you'll be able to converse in real time with people who do not know your natural language.^{1, 2} Some smartphones, can now work together with in ear headphones to provide near-live translation of many languages.^{3, 4, 5} In the "IBM Watson and Cognitive Computing" chapter, we develop a script that does near real-time inter-language translation among languages supported by Watson.

¹ <https://www.skype.com/en/features/skype-translator/>.

² <https://www.microsoft.com/en-us/translator/business/live/>.

³ <https://www.telegraph.co.uk/technology/2017/10/04/googles-new-headphones-can-translate-foreign-languages-real/>.

⁴ https://store.google.com/us/product/google_pixel_buds?hl=en-US.

⁵ <http://www.chicagotribune.com/bluesky/originals/ct-bsi-google-pixel-buds-review-20171115-story.html>.

The TextBlob library uses Google Translate to detect a text's language and translate TextBlobs, Sentences and Words into other languages.⁶ Let's use **detect_language method** to detect the language of the text we're manipulating ('en' is English):

⁶These features require an Internet connection.

[lick here to view code image](#)

```
In [22]: blob
Out[22]: TextBlob("Today is a beautiful day. Tomorrow looks like bad weather.")

In [23]: blob.detect_language()
Out[23]: 'en'
```

Next, let's use the **translate method** to translate the text to Spanish ('es') then detect the language on the result. The to keyword argument specifies the target language.

[lick here to view code image](#)

```
In [24]: spanish = blob.translate(to='es')

In [25]: spanish
Out[25]: TextBlob("Hoy es un hermoso dia. Mañana parece mal tiempo.")
```

```
In [26]: spanish.detect_language()
Out[26]: 'es'
```

Next, let's translate our `TextBlob` to simplified Chinese (specified as `'zh'` or `'zh-CN'`) then detect the language on the result:

[lick here to view code image](#)

```
In [27]: chinese = blob.translate(to='zh')

In [28]: chinese
Out[28]: TextBlob("")

In [29]: chinese.detect_language()
Out[29]: 'zh-CN'
```

Method `detect_language`'s output always shows simplified Chinese as `'zh-CN'`, even though the `translate` function can receive simplified Chinese as `'zh'` or `'zh-CN'`.

In each of the preceding cases, Google Translate automatically detects the source language. You can specify a source language explicitly by passing the `from_lang` keyword argument to the `translate` method, as in

[lick here to view code image](#)

```
chinese = blob.translate(from_lang='en', to='zh')
```

Google Translate uses iso-639-1 ⁷ language codes listed at

⁷ SO is the International Organization for Standardization (<https://www.iso.org/>).

https://en.wikipedia.org/wiki/List_of_ISO_639-1_codes

For the supported languages, you'd use these codes as the values of the `from_lang` and `to` keyword arguments. Google Translate's list of supported languages is at:

<https://cloud.google.com/translate/docs/languages>

Calling `translate` without arguments translates from the detected source language to English:

[lick here to view code image](#)

```
In [30]: spanish.translate()
Out[30]: TextBlob("Today is a beautiful day. Tomorrow    seems like bad weather.

In [31]: chinese.translate()
Out[31]: TextBlob("Today is a beautiful day. Tomorrow    looks like bad weather.
```



note the slight difference in the English results.

11.2.8 Inflection: Pluralization and Singularization

Inflections are different forms of the same words, such as singular and plural (like “person” and “people”) and different verb tenses (like “run” and “ran”). When you’re calculating word frequencies, you might first want to convert all inflected words to the same form for more accurate word frequencies. `Words` and `WordLists` each support converting words to their singular or plural forms. Let’s pluralize and singularize a couple of `Word` objects:

[lick here to view code image](#)

```
In [1]: from textblob import Word

In [2]: index = Word('index')

In [3]: index.pluralize()
Out[3]: 'indices'

In [4]: cacti = Word('cacti')

In [5]: cacti.singularize()
Out[5]: 'cactus'
```

Pluralizing and singularizing are sophisticated tasks which, as you can see above, are not as simple as adding or removing an “s” or “es” at the end of a word.

You can do the same with a `WordList`:

[lick here to view code image](#)

```
In [6]: from textblob import TextBlob

In [7]: animals = TextBlob('dog cat fish bird').words

In [8]: animals.pluralize()
Out[8]: WordList(['dogs', 'cats', 'fish', 'birds'])
```

Note that the word “fish” is the same in both its singular and plural forms.

11.2.9 Spell Checking and Correction

For natural language processing tasks, it’s important that the text be free of spelling errors. Software packages for writing and editing text, like Microsoft Word, Google Docs and others automatically check your spelling as you type and typically display a red line under misspelled words. Other tools enable you to manually invoke a spelling checker.

You can check a `Word`’s spelling with its **`spellcheck method`**, which returns a list of tuples containing possible correct spellings and a confidence value. Let’s assume we meant to type the word “they” but we misspelled it as “theyr.” The spell checking results show two possible corrections with the word ‘they’ having the highest confidence value:

[lick here to view code image](#)

```
In [1]: from textblob import Word

In [2]: word = Word('theyr')
```

```
In [3]: %precision 2
Out[3]: '%.2f'

In [4]: word.spellcheck()
Out[4]: [('they', 0.57), ('their', 0.43)]
```

Note that the word with the highest confidence value might *not* be the correct word for the given context.

TextBlobs, Sentences and Words all have a **correct method** that you can call to correct spelling. Calling `correct` on a `Word` returns the correctly spelled word that has the highest confidence value (as returned by `spellcheck`):

[lick here to view code image](#)

```
In [5]: word.correct() # chooses word with the highest confidence value
Out[5]: 'they'
```

Calling `correct` on a `TextBlob` or `Sentence` checks the spelling of each word. For each incorrect word, `correct` replaces it with the correctly spelled one that has the highest confidence value:

[lick here to view code image](#)

```
In [6]: from textblob import Word

In [7]: sentence = TextBlob('Ths sentence has misspelled wrds.')

In [8]: sentence.correct()
Out[8]: TextBlob("The sentence has misspelled words.")
```

11.2.10 Normalization: Stemming and Lemmatization

Stemming removes a prefix or suffix from a word leaving only a stem, which may or may not be a real word. **Lemmatization** is similar, but factors in the word's part of speech and meaning and results in a real word.

Stemming and lemmatization are **normalization** operations, in which you prepare words for analysis. For example, before calculating statistics on words in a body of text, you might convert all words to lowercase so that capitalized and lowercase words are not treated differently. Sometimes, you might want to use a word's root to represent the word's many forms. For example, in a given application, you might want to treat all of the following words as "program": program, programs, programmer, programming and programmed (and perhaps U.K. English spellings, like programmes as well).

Words and WordLists each support stemming and lemmatization via the methods **stem** and **lemmatize**. Let's use both on a `Word`:

[lick here to view code image](#)

```
In [1]: from textblob import Word
```

```
In [2]: word = Word('varieties')

In [3]: word.stem()
Out[3]: 'varieti'

In [4]: word.lemmatize()
Out[4]: 'variety'
```

11.2.11 Word Frequencies

Various techniques for detecting similarity between documents rely on word frequencies. As you'll see here, `TextBlob` automatically counts word frequencies. First, let's load the e-book for Shakespeare's *Romeo and Juliet* into a `TextBlob`. To do so, we'll use the **`Path` class** from the Python Standard Library's **`pathlib` module**:

[lick here to view code image](#)

```
In [1]: from pathlib import Path

In [2]: from textblob import TextBlob

In [3]: blob = TextBlob(Path('RomeoAndJuliet.txt').read_text())
```

Use the file `RomeoAndJuliet.txt`⁸ that you downloaded earlier. We assume here that you started your IPython session from that folder. When you read a file with `Path`'s **`read_text` method**, it closes the file immediately after it finishes reading the file.

⁸Each Project Gutenberg e-book includes additional text, such as their licensing information, that's not part of the e-book itself. For this example, we used a text editor to remove that text from our copy of the e-book.

You can access the word frequencies through the `TextBlob`'s **`word_counts` dictionary**. Let's get the counts of several words in the play:

[lick here to view code image](#)

```
In [4]: blob.word_counts['juliet']
Out[4]: 190

In [5]: blob.word_counts['romeo']
Out[5]: 315

In [6]: blob.word_counts['thou']
Out[6]: 278
```

If you already have tokenized a `TextBlob` into a `WordList`, you can count specific words in the list via the **`count` method**:

[lick here to view code image](#)

```
In [7]: blob.words.count('joy')
Out[7]: 14

In [8]: blob.noun_phrases.count('lady capulet')
```

11.2.12 Getting Definitions, Synonyms and Antonyms from WordNet

WordNet⁹ is a word database created by Princeton University. The TextBlob library uses the NLTK library's WordNet interface, enabling you to look up word definitions, and get synonyms and antonyms. For more information, check out the NLTK WordNet interface documentation at:

⁹ <https://wordnet.princeton.edu/>.

<https://www.nltk.org/api/nltk.corpus.reader.html#module-nltk.corpus.reader.wordnet>

Getting Definitions

First, let's create a Word:

[lick here to view code image](#)

```
In [1]: from textblob import Word

In [2]: happy = Word('happy')
```

The Word class's **definitions property** returns a list of all the word's definitions in the WordNet database:

[lick here to view code image](#)

```
In [3]: happy.definitions
Out[3]:
['enjoying or showing or marked by joy or pleasure',
 'marked by good fortune',
 'eagerly disposed to act or to be of service',
 'well expressed and to the point']
```

The database does not necessarily contain every dictionary definition of a given word. There's also a **define method** that enables you to pass a part of speech as an argument so you can get definitions matching only that part of speech.

Getting Synonyms

You can get a Word's **synsets**—that is, its sets of synonyms—via the **synsets property**. The result is a list of Synset objects:

```
In [4]: happy.synsets
Out[4]:
[Synset('happy.a.01'),
 Synset('felicitous.s.02'),
 Synset('glad.s.02'),
 Synset('happy.s.04')]
```

Each Synset represents a group of synonyms. In the notation `happy.a.01`:

- `happy` is the original Word's lemmatized form (in this case, it's the same).
- `a` is the part of speech, which can be `a` for adjective, `n` for noun, `v` for verb, `r` for adverb or `s` for adjective satellite. Many adjective synsets in WordNet have satellite synsets that represent similar adjectives.
- `01` is a 0-based index number. Many words have multiple meanings, and this is the index number of the corresponding meaning in the WordNet database.

There's also a **`get_synsets` method** that enables you to pass a part of speech as an argument so you can get `Synsets` matching only that part of speech.

You can iterate through the `synsets` list to find the original word's synonyms. Each `Synset` has a **`lemmas` method** that returns a list of `Lemma` objects representing the synonyms. A `Lemma`'s `name` method returns the synonymous word as a string. In the following code, for each `Synset` in the `synsets` list, the nested `for` loop iterates through that `Synset`'s `Lemmas` (if any). Then we add the synonym to the set named `synonyms`. We used a set collection because it automatically eliminates any duplicates we add to it:

[lick here to view code image](#)

```
In [5]: synonyms = set()

In [6]: for synset in happy.synsets:
...:     for lemma in synset.lemmas():
...:         synonyms.add(lemma.name())
...:

In [7]: synonyms
Out[7]: {'felicitous', 'glad', 'happy', 'well-chosen'}
```

Getting Antonyms

If the word represented by a `Lemma` has antonyms in the WordNet database, invoking the `Lemma`'s `antonyms` method returns a list of `Lemmas` representing the antonyms (or an empty list if there are no antonyms in the database). In snippet [4] you saw there were four `Synsets` for 'happy'. First, let's get the `Lemmas` for the `Synset` at index 0 of the `synsets` list:

[lick here to view code image](#)

```
In [8]: lemmas = happy.synsets[0].lemmas()

In [9]: lemmas
Out[9]: [Lemma('happy.a.01.happy')]
```

In this case, `lemmas` returned a list of one `Lemma` element. We can now check whether the database has any corresponding antonyms for that `Lemma`:

[lick here to view code image](#)

```
In [10]: lemmas[0].antonyms()
```



```
Out[10]: [Lemma('unhappy.a.01.unhappy')]
```

The result is list of Lemmas representing the antonym(s). Here, we see that the one antonym for 'happy' in the database is 'unhappy'.

11.2.13 Deleting Stop Words

Stop words are common words in text that are often removed from text before analyzing it because they typically do not provide useful information. The following table shows NLTK's list of English stop words, which is returned by the NLTK `stopwords` module's `words` function^o (which we'll use momentarily):

^o <https://www.nltk.org/book/ch02.html>.

NLTK's English stop words list

```
['a', 'about', 'above', 'after', 'again', 'against', 'ain',  
'all', 'am', 'an', 'and', 'any', 'are', 'aren', "aren't",  
'as', 'at', 'be', 'because', 'been', 'before', 'being',  
'below', 'between', 'both', 'but', 'by', 'can', 'couldn',  
"couldn't", 'd', 'did', 'didn', "didn't", 'do', 'does',  
'doesn', "doesn't", 'doing', 'don', "don't", 'down', 'during',  
'each', 'few', 'for', 'from', 'further', 'had', 'hadn',  
"hadn't", 'has', 'hasn', "hasn't", 'have', 'haven', "haven't",  
'having', 'he', 'her', 'here', 'hers', 'herself', 'him',  
'himself', 'his', 'how', 'i', 'if', 'in', 'into', 'is', 'isn',  
"isn't", 'it', "it's", 'its', 'itself', 'just', 'll', 'm',  
'ma', 'me', 'mightn', "mightn't", 'more', 'most', 'mustn',  
"mustn't", 'my', 'myself', 'needn', "needn't", 'no', 'nor',  
'not', 'now', 'o', 'of', 'off', 'on', 'once', 'only', 'or',  
'other', 'our', 'ours', 'ourselves', 'out', 'over', 'own',  
're', 's', 'same', 'shan', "shan't", 'she', "she's", 'should',  
"should've", 'shouldn', "shouldn't", 'so', 'some', 'such',  
't', 'than', 'that', "that'll", 'the', 'their', 'theirs',  
'them', 'themselves', 'then', 'there', 'these', 'they',  
'this', 'those', 'through', 'to', 'too', 'under', 'until',  
'up', 've', 'very', 'was', 'wasn', "wasn't", 'we', 'were',  
'weren', "weren't", 'what', 'when', 'where', 'which', 'while',  
'who', 'whom', 'why', 'will', 'with', 'won', "won't",  
'wouldn', "wouldn't", 'y', 'you', "you'd", "you'll", "you're",  
"you've", 'your', 'yours', 'yourself', 'yourselves']
```

The NLTK library has lists of stop words for several other natural languages as well. Before using NLTK's stop-words lists, you must download them, which you do with the `nltk`

module's **download function**:

[lick here to view code image](#)

```
In [1]: import nltk

In [2]: nltk.download('stopwords')
[nltk_data] Downloading package stopwords to
[nltk_data] C:\Users\PaulDeitel\AppData\Roaming\nltk_data...
[nltk_data] Unzipping corpora\stopwords.zip.
Out[2]: True
```

For this example, we'll load the 'english' stop words list. First import stopwords from the `nltk.corpus` module, then use `stopwords` method words to load the 'english' stop words list:

[lick here to view code image](#)

```
In [3]: from nltk.corpus import stopwords

In [4]: stops = stopwords.words('english')
```

Next, let's create a `TextBlob` from which we'll remove stop words:

[lick here to view code image](#)

```
In [5]: from textblob import TextBlob
In [6]: blob = TextBlob('Today is a beautiful day.')
```

Finally, to remove the stop words, let's use the `TextBlob`'s words in a list comprehension that adds each word to the resulting list only if the word is not in `stops`:

[lick here to view code image](#)

```
In [7]: [word for word in blob.words if word not in stops]
Out[7]: ['Today', 'beautiful', 'day']
```

11.2.14 n-grams

An **n-gram**¹ is a sequence of n text items, such as letters in words or words in a sentence. In natural language processing, n -grams can be used to identify letters or words that frequently appear adjacent to one another. For text-based user input, this can help predict the next letter or word a user will type—such as when completing items in IPython with tab-completion or when entering a message to a friend in your favorite smartphone messaging app. For speech-to-text, n -grams might be used to improve the quality of the transcription. N-grams are a form of **co-occurrence** in which words or letters appear near each other in a body of text.

¹ <https://en.wikipedia.org/wiki/N-gram>.

`TextBlob`'s **ngrams** method produces a list of `WordList` n -grams of length three by

default—known as *trigrams*. You can pass the keyword argument `n` to produce n -grams of any desired length. The output shows that the first trigram contains the first three words in the sentence ('Today', 'is' and 'a'). Then, `ngrams` creates a trigram starting with the second word ('is', 'a' and 'beautiful') and so on until it creates a trigram containing the last three words in the `TextBlob`:

[lick here to view code image](#)

```
In [1]: from textblob import TextBlob

In [2]: text = 'Today is a beautiful day. Tomorrow looks like bad weather.'

In [3]: blob = TextBlob(text)

In [4]: blob.ngrams()
Out[4]:
[WordList(['Today', 'is', 'a']),
 WordList(['is', 'a', 'beautiful']),
 WordList(['a', 'beautiful', 'day']),
 WordList(['beautiful', 'day', 'Tomorrow']),
 WordList(['day', 'Tomorrow', 'looks']),
 WordList(['Tomorrow', 'looks', 'like']),
 WordList(['looks', 'like', 'bad']),
 WordList(['like', 'bad', 'weather'])]
```

The following produces n -grams consisting of five words:

[lick here to view code image](#)

```
In [5]: blob.ngrams(n=5)
Out[5]:
[WordList(['Today', 'is', 'a', 'beautiful', 'day']),
 WordList(['is', 'a', 'beautiful', 'day', 'Tomorrow']),
 WordList(['a', 'beautiful', 'day', 'Tomorrow', 'looks']),
 WordList(['beautiful', 'day', 'Tomorrow', 'looks', 'like']),
 WordList(['day', 'Tomorrow', 'looks', 'like', 'bad']),
 WordList(['Tomorrow', 'looks', 'like', 'bad', 'weather'])]
```

11.3 VISUALIZING WORD FREQUENCIES WITH BAR CHARTS AND WORD CLOUDS

Earlier, we obtained frequencies for a few words in *Romeo and Juliet*. Sometimes frequency visualizations enhance your corpus analyses. There's often more than one way to visualize data, and sometimes one is superior to others. For example, you might be interested in word frequencies relative to one another, or you may just be interested in relative uses of words in a corpus. In this section, we'll look at two ways to visualize word frequencies:

- A bar chart that *quantitatively* visualizes the top 20 words in *Romeo and Juliet* as bars representing each word and its frequency.
- A **word cloud** that *qualitatively* visualizes more frequently occurring words in bigger fonts and less frequently occurring words in smaller fonts.

11.3.1 Visualizing Word Frequencies with Pandas

Let's visualize *Romeo and Juliet*'s top 20 words that are *not* stop words. To do this, we'll use features from TextBlob, NLTK and pandas. Pandas visualization capabilities are based on Matplotlib, so launch IPython with the following command for this session:

```
ipython --matplotlib
```

Loading the Data

First, let's load *Romeo and Juliet*. Launch IPython from the `ch11` examples folder before executing the following code so you can access the e-book file `RomeoAndJuliet.txt` that you downloaded earlier in the chapter:

[lick here to view code image](#)

```
In [1]: from pathlib import Path

In [2]: from textblob import TextBlob

In [3]: blob = TextBlob(Path('RomeoAndJuliet.txt').read_text())
```

Next, load the NLTK stopwords:

[lick here to view code image](#)

```
In [4]: from nltk.corpus import stopwords

In [5]: stop_words = stopwords.words('english')
```

Getting the Word Frequencies

To visualize the top 20 words, we need each word and its frequency. Let's call the `blob.word_counts` dictionary's `items` method to get a list of word-frequency tuples:

[lick here to view code image](#)

```
In [6]: items = blob.word_counts.items()
```

Eliminating the Stop Words

Next, let's use a list comprehension to eliminate any tuples containing stop words:

[lick here to view code image](#)

```
In [7]: items = [item for item in items if item[0] not in stop_words]
```

The expression `item[0]` gets the word from each tuple so we can check whether it's in `stop_words`.

Sorting the Words by Frequency

To determine the top 20 words, let's sort the tuples in `items` in descending order by

frequency. We can use built-in function `sorted` with a `key` argument to sort the tuples by the frequency element in each tuple. To specify the tuple element to sort by, use the **itemgetter function** from the Python Standard Library's **operator module**:

[lick here to view code image](#)

```
In [8]: from operator import itemgetter

In [9]: sorted_items = sorted(items, key=itemgetter(1), reverse=True)
```

As `sorted` orders items' elements, it accesses the element at index 1 in each tuple via the expression `itemgetter(1)`. The `reverse=True` keyword argument indicates that the tuples should be sorted in *descending* order.

Getting the Top 20 Words

Next, we use a slice to get the top 20 words from `sorted_items`. When `TextBlob` tokenizes a corpus, it splits all contractions at their apostrophes and counts the total number of apostrophes as one of the “words.” *Romeo and Juliet* has many contractions. If you display `sorted_items[0]`, you'll see that they are the most frequently occurring “word” with 867 of them.² We want to display only words, so we ignore element 0 and get a slice containing elements 1 through 20 of `sorted_items`:

²In some locales this does not happen and element 0 is indeed 'romeo'.

[lick here to view code image](#)

```
In [10]: top20 = sorted_items[1:21]
```

Convert top20 to a DataFrame

Next, let's convert the `top20` list of tuples to a pandas `DataFrame` so we can visualize it conveniently:

[lick here to view code image](#)

```
In [11]: import pandas as pd

In [12]: df = pd.DataFrame(top20, columns=['word', 'count'])

In [13]: df
Out[13]:
```

	word	count
0	romeo	315
1	thou	278
2	juliet	190
3	thy	170
4	capulet	163
5	nurse	149
6	love	148
7	thee	138
8	lady	117
9	shall	110
10	friar	105
11	come	94

```
12 mercutio      88
13 lawrence     82
14      good     80
15 benvolio     79
16      tybalt   79
17      enter    75
18      go       75
19      night    73
```

Visualizing the DataFrame

To visualize the data, we'll use the **bar method** of the DataFrame's **plot property**. The arguments indicate which column's data should be displayed along the *x*- and *y*-axes, and that we do not want to display a legend on the graph:

[lick here to view code image](#)

```
In [14]: axes = df.plot.bar(x='word', y='count', legend=False)
```

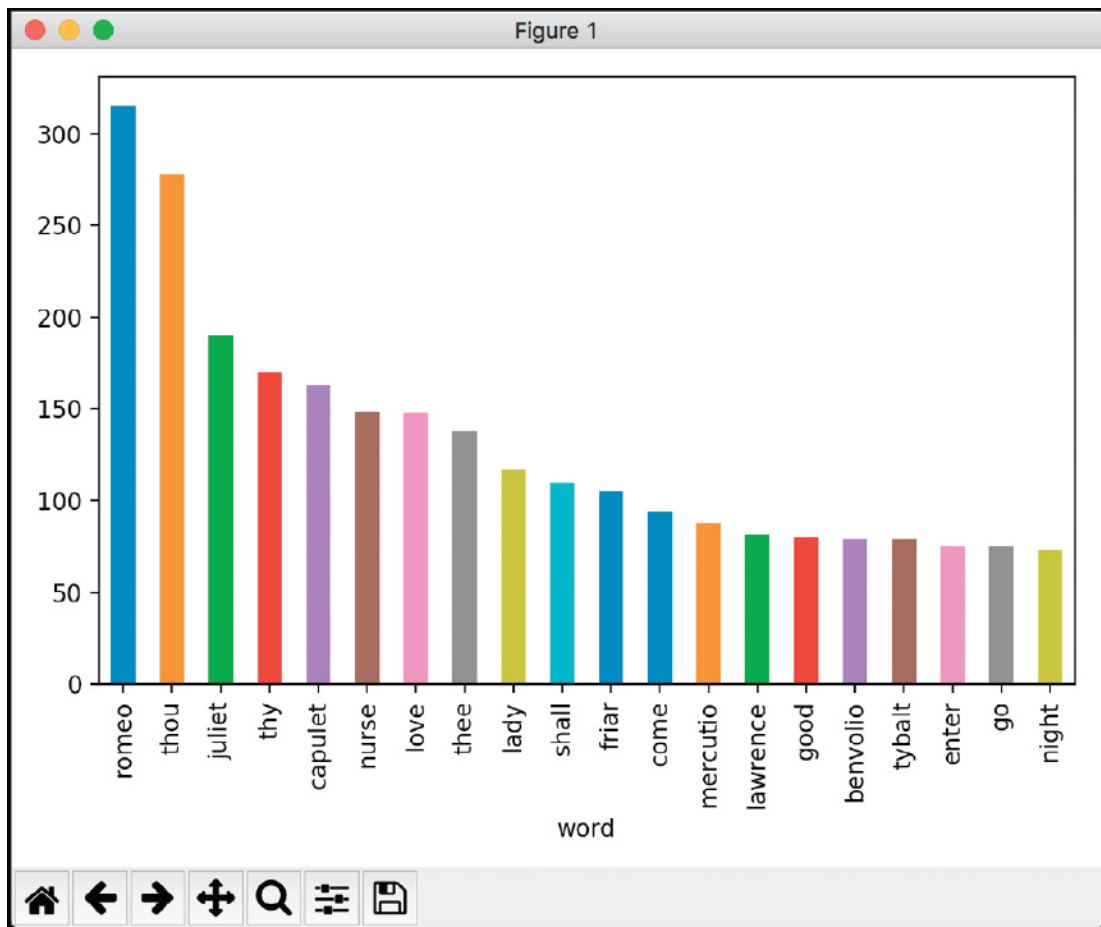
The `bar` method creates and displays a Matplotlib bar chart.

When you look at the initial bar chart that appears, you'll notice that some of the words are truncated. To fix that, use Matplotlib's `gcf` (get current figure) function to get the Matplotlib figure that pandas displayed, then call the figure's `tight_layout` method. This compresses the bar chart to ensure all its components fit:

[lick here to view code image](#)

```
In [15]: import matplotlib.pyplot as plt
In [16]: plt.gcf().tight_layout()
```

The final graph is shown below:



11.3.2 Visualizing Word Frequencies with Word Clouds

Next, we'll build a word cloud that visualizes the top 200 words in *Romeo and Juliet*. You can use the open source **wordcloud module's** ³ **WordCloud class** to generate word clouds with just a few lines of code. By default, wordcloud creates rectangular word clouds, but as you'll see the library can create word clouds with arbitrary shapes.

³ https://github.com/amueller/word_cloud.

Installing the wordcloud Module

To install wordcloud, open your Anaconda Prompt (Windows), Terminal (macOS/Linux) or shell (Linux) and enter the command:

```
conda install -c conda-forge wordcloud
```

Windows users might need to run the Anaconda Prompt as an Administrator for proper software installation privileges. To do so, right-click Anaconda Prompt in the start menu and select **More > Run as administrator**.

Loading the Text

First, let's load *Romeo and Juliet*. Launch IPython from the ch11 examples folder before executing the following code so you can access the e-book file `RomeoAndJuliet.txt` you downloaded earlier:

[lick here to view code image](#)

```
In [1]: from pathlib import Path
```

```
In [2]: text = Path('RomeoAndJuliet.txt').read_text()
```

Loading the Mask Image that Specifies the Word Cloud's Shape

To create a word cloud of a given shape, you can initialize a `WordCloud` object with an image known as a *mask*. The `WordCloud` fills non-white areas of the mask image with text. We'll use a heart shape in this example, provided as `mask_heart.png` in the `ch11` examples folder. More complex masks require more time to create the word cloud.

Let's load the mask image by using the **`imread` function** from the `imageio` module that comes with Anaconda:

[lick here to view code image](#)

```
In [3]: import imageio

In [4]: mask_image = imageio.imread('mask_heart.png')
```

This function returns the image as a NumPy array, which is required by `WordCloud`.

Configuring the WordCloud Object

Next, let's create and configure the `WordCloud` object:

[lick here to view code image](#)

```
In [5]: from wordcloud import WordCloud

In [6]: wordcloud = WordCloud(colormap='prism', mask=mask_image,
...:                          background_color='white')
...:
```

The default `WordCloud` width and height in pixels is 400x200, unless you specify `width` and `height` keyword arguments or a mask image. For a mask image, the `WordCloud` size is the image's size. `WordCloud` uses Matplotlib under the hood. `WordCloud` assigns random colors from a color map. You can supply the `colormap` keyword argument and use one of Matplotlib's named color maps. For a list of color map names and their colors, see:

https://matplotlib.org/examples/color/colormaps_reference.html

The `mask` keyword argument specifies the `mask_image` we loaded previously. By default, the word is drawn on a black background, but we customized this with the `background_color` keyword argument by specifying a 'white' background. For a complete list of `WordCloud`'s keyword arguments, see

http://amueller.github.io/word_cloud/generated/wordcloud.WordCloud.html

Generating the Word Cloud

`WordCloud`'s **`generate` method** receives the text to use in the word cloud as an argument and creates the word cloud, which it returns as a `WordCloud` object:

[lick here to view code image](#)

```
In [7]: wordcloud = wordcloud.generate(text)
```

Before creating the word cloud, `generate` first removes stop words from the text argument using the `wordcloud` module's built-in stop-words list. Then `generate` calculates the word frequencies for the remaining words. The method uses a maximum of 200 words in the word cloud by default, but you can customize this with the `max_words` keyword argument.

Saving the Word Cloud as an Image File

Finally, we use WordCloud's **to_file method** to save the word cloud image into the specified file:

[lick here to view code image](#)

```
In [8]: wordcloud = wordcloud.to_file('RomeoAndJulietHeart.png')
```

You can now go to the `ch11` examples folder and double-click the `RomeoAndJuliet.png` image file on your system to view it—your version might have the words in different positions and different colors:



Generating a Word Cloud from a Dictionary

If you already have a dictionary of key–value pairs representing word counts, you can pass it to WordCloud’s **fit_words** method. This method assumes you’ve already removed the stop words.

Displaying the Image with Matplotlib

If you'd like to display the image on the screen, you can use the IPython magic

```
%matplotlib
```

to enable interactive Matplotlib support in IPython, then execute the following statements:

[lick here to view code image](#)

```
import matplotlib.pyplot as plt
plt.imshow(wordcloud)
```

11.4 READABILITY ASSESSMENT WITH TEXTATISTIC

An interesting use of natural language processing is assessing text **readability**, which is affected by the vocabulary used, sentence structure, sentence length, topic and more. While writing this book, we used the paid tool Grammarly to help tune the writing and ensure the text's readability for a wide audience.

In this section, we'll use the **Textatistic library**⁴ to assess readability.⁵ There are many formulas used in natural language processing to calculate readability. Textatistic uses five popular readability formulas—Flesch Reading Ease, Flesch-Kincaid, Gunning Fog, Simple Measure of Gobbledygook (SMOG) and Dale-Chall.

⁴ <https://github.com/erinhengel/Textatistic>.

⁵Some other Python readability assessment libraries include readability-score, textstat, readability and pylinguistics.

Install Textatistic

To install Textatistic, open your Anaconda Prompt (Windows), Terminal (macOS/Linux) or shell (Linux), then execute the following command:

```
pip install textatistic
```

Windows users might need to run the Anaconda Prompt as an Administrator for proper software installation privileges. To do so, right-click Anaconda Prompt in the start menu and select **More > Run as administrator**.

Calculating Statistics and Readability Scores

First, let's load *Romeo and Juliet* into the `text` variable:

[lick here to view code image](#)

```
In [1]: from pathlib import Path

In [2]: text = Path('RomeoAndJuliet.txt').read_text()
```

Calculating statistics and readability scores requires a **Textatistic** object that's initialized with the text you want to assess:

[lick here to view code image](#)

```
In [3]: from textatistic import Textatistic

In [4]: readability = Textatistic(text)
```

`Textatistic` method `dict` returns a dictionary containing various statistics and the readability scores ⁶:

⁶Each Project Gutenberg e-book includes additional text, such as their licensing information, that's not part of the e-book itself. For this example, we used a text editor to remove that text from our copy of the e-book.

[lick here to view code image](#)

```
In [5]: %precision 3
Out[5]: '0.3f'

In [6]: readability.dict()
Out[6]:
{'char_count': 115141,
 'word_count': 26120,
 'sent_count': 3218,
 'sybl_count': 30166,
 'notdalechall_count': 5823,
 'polysyblword_count': 549,
 'flesch_score': 100.892,
 'fleschkincaid_score': 1.203,
 'gunningfog_score': 4.087,
 'smog_score': 5.489,
 'dalechall_score': 7.559}
```

Each of the values in the dictionary is also accessible via a `Textatistic` property of the same name as the keys shown in the preceding output. The statistics produced include:

- `char_count`—The number of characters in the text.
- `word_count`—The number of words in the text.
- `sent_count`—The number of sentences in the text.
- `sybl_count`—The number of syllables in the text.
- `notdalechall_count`—A count of the words that are not on the Dale-Chall list, which is a list of words understood by 80% of 5th graders. ⁷ The higher this number is compared to the total word count, the less readable the text is considered to be.

⁷ <http://www.readabilityformulas.com/articles/dale-chall-readability-word-list.php>.

- `polysyblword_count`—The number of words with three or more syllables.
- `flesch_score`—The Flesch Reading Ease score, which can be mapped to a grade level. Scores over 90 are considered readable by 5th graders. Scores under 30 require a college degree. Ranges in between correspond to the other grade levels.
- `fleschkincaid_score`—The Flesch-Kincaid score, which corresponds to a specific grade level.
- `gunningfog_score`—The Gunning Fog index value, which corresponds to a specific

grade level.

- `smog_score`—The Simple Measure of Gobbledygook (SMOG), which corresponds to the years of education required to understand text. This measure is considered particularly effective for healthcare materials.⁸

⁸ <https://en.wikipedia.org/wiki/SMOG>.

- `dalechall_score`—The Dale-Chall score, which can be mapped to grade levels from 4 and below to college graduate (grade 16) and above. This score considered to be most reliable for a broad range of text types.^{9, 0}

⁹ https://en.wikipedia.org/wiki/Readability#The_Dale%E2%80%93Chall_formula.

⁰ <http://www.readabilityformulas.com/articles/how-do-i-decide-high-readability-formula-to-use.php>.

For more details on each of the readability scores produced here and several others, see

<https://en.wikipedia.org/wiki/Readability>

The Textastic documentation also shows the readability formulas used:

<http://www.erinhengel.com/software/textastic/>

11.5 NAMED ENTITY RECOGNITION WITH SPACY

NLP can determine what a text is about. A key aspect of this is **named entity recognition**, which attempts to locate and categorize items like dates, times, quantities, places, people, things, organizations and more. In this section, we'll use the named entity recognition capabilities in the **spaCy NLP library**^{1, 2} to analyze text.

¹ <https://spacy.io/>.

²You may also want to check out Textacy (<https://github.com/chartbeat-abs/textacy>) an NLP library built on spaCy that supports additional NLP tasks.

Install spaCy

To install spaCy, open your Anaconda Prompt (Windows), Terminal (macOS/Linux) or shell (Linux), then execute the following command:

[lick here to view code image](#)

```
conda install -c conda-forge spacy
```

Windows users might need to run the Anaconda Prompt as an Administrator for proper software installation privileges. To do so, right-click Anaconda Prompt in the start menu and select **More > Run as administrator**.

Once the install completes, you also need to execute the following command, so spaCy can

download additional components it needs for processing English (en) text:

[lick here to view code image](#)

```
python -m spacy download en
```

Loading the Language Model

The first step in using spaCy is to load the language model representing the natural language of the text you're analyzing. To do this, you'll call the `spacy` module's **load function**. Let's load the English model that we downloaded above:

[lick here to view code image](#)

```
In [1]: import spacy

In [2]: nlp = spacy.load('en')
```

The spaCy documentation recommends the variable name `nlp`.

Creating a spaCy Doc

Next, you use the `nlp` object to create a spaCy **Doc** object ³ representing the document to process. Here we used a sentence from the introduction to the World Wide Web in many of our books:

³ <https://spacy.io/api/doc>.

[lick here to view code image](#)

```
In [3]: document = nlp('In 1994, Tim Berners-Lee founded the ' +
...:      'World Wide Web Consortium (W3C), devoted to ' +
...:      'developing web technologies')
...:
```

Getting the Named Entities

The `Doc` object's **ents property** returns a tuple of **Span** objects representing the named entities found in the `Doc`. Each `Span` has many properties. ⁴ Let's iterate through the `Spans` and display the `text` and `label_` properties:

⁴ <https://spacy.io/api/span>.

[lick here to view code image](#)

```
In [4]: for entity in document.ents:
...:     print(f'{entity.text}: {entity.label_}')
...:
1994: DATE
Tim Berners-Lee: PERSON
the World Wide Web Consortium: ORG
```

Each Span's **text property** returns the entity as a string, and the **label_property** returns a string indicating the entity's kind. Here, spaCy found three entities representing a DATE (1994), a PERSON (Tim Berners-Lee) and an ORG (organization; the World Wide Web Consortium). For more spaCy information and to take a look at its Quickstart guide, see

```
https://spacy.io/usage/models#section-quickstart
```

11.6 SIMILARITY DETECTION WITH SPACY

Similarity detection is the process of analyzing documents to determine how alike they are. One possible similarity detection technique is word frequency counting. For example, some people believe that the works of William Shakespeare actually might have been written by Sir Francis Bacon, Christopher Marlowe or others.⁵ Comparing the word frequencies of their works with those of Shakespeare can reveal writing-style similarities.

⁵ https://en.wikipedia.org/wiki/Shakespeare_authorship_question.

Various machine-learning techniques we'll discuss in later chapters can be used to study document similarity. However, as is often the case in Python, there are libraries such as spaCy and Gensim that can do this for you. Here, we'll use spaCy's similarity detection features to compare Doc objects representing Shakespeare's *Romeo and Juliet* with Christopher Marlowe's *Edward the Second*. You can download *Edward the Second* from Project Gutenberg as we did for *Romeo and Juliet* earlier in the chapter.⁶

⁶Each Project Gutenberg e-book includes additional text, such as their licensing information, that's not part of the e-book itself. For this example, we used a text editor to remove that text from our copies of the e-books.

Loading the Language Model and Creating a spaCy Doc

As in the preceding section, we first load the English model:

[lick here to view code image](#)

```
In [1]: import spacy

In [2]: nlp = spacy.load('en')
```

Creating the spaCy Docs

Next, we create two Doc objects—one for *Romeo and Juliet* and one for *Edward the Second*:

[lick here to view code image](#)

```
In [3]: from pathlib import Path

In [4]: document1 = nlp(Path('RomeoAndJuliet.txt').read_text())

In [5]: document2 = nlp(Path('EdwardTheSecond.txt').read_text())
```

Comparing the Books' Similarity

Finally, we use the `Doc` class's **similarity method** to get a value from 0.0 (not similar) to 1.0 (identical) indicating how similar the documents are:

[lick here to view code image](#)

```
In [6]: document1.similarity(document2)
Out[6]: 0.9349950179100041
```

spaCy believes these two documents have significant similarities. For comparison purposes, we created a `Doc` representing a current news story and compared it with *Romeo and Juliet*. As expected, spaCy returned a low value indicating little similarity between them. Try copying a current news article into a text file, then performing the similarity comparison yourself.

11.7 OTHER NLP LIBRARIES AND TOOLS

We've shown you various NLP libraries, but it's always a good idea to investigate the range of options available to you so you can leverage the best tools for your tasks. Below are some additional mostly free and open source NLP libraries and APIs:

- Gensim—Similarity detection and topic modeling.
- Google Cloud Natural Language API—Cloud-based API for NLP tasks such as named entity recognition, sentiment analysis, parts-of-speech analysis and visualization, determining content categories and more.
- Microsoft Linguistic Analysis API.
- Bing sentiment analysis—Microsoft's Bing search engine now uses sentiment in its search results. At the time of this writing, sentiment analysis in search results is available only in the United States.
- PyTorch NLP—Deep learning library for NLP.
- Stanford CoreNLP—A Java NLP library, which also provides a Python wrapper. Includes coreference resolution, which finds all references to the same thing.
- Apache OpenNLP—Another Java-based NLP library for common tasks, including coreference resolution. Python wrappers are available.
- PyNLPl (pineapple)—Python NLP library, includes basic and more sophisticated NLP capabilities.
- SnowNLP—Python library that simplifies Chinese text processing.
- KoNLPy—Korean language NLP.
- `stop-words`—Python library with stop words for many languages. We used NLTK's stop words lists in this chapter.

- TextRazor—A paid cloud-based NLP API that provides a free tier.

11.8 MACHINE LEARNING AND DEEP LEARNING NATURAL LANGUAGE APPLICATIONS

There are many natural language applications that require machine learning and deep learning techniques. We'll discuss some of the following in our machine learning and deep learning chapters:

- Answering natural language questions—For example, our publisher Pearson Education, has a partnership with IBM Watson that uses Watson as a virtual tutor. Students ask Watson natural language questions and get answers.
- Summarizing documents—analyzing documents and producing short summaries (also called abstracts) that can, for example, be included with search results and can help you decide what to read.
- Speech synthesis (speech-to-text) and speech recognition (text-to-speech)—We use these in our “IBM Watson” chapter, along with inter-language text-to-text translation, to develop a near real-time inter-language voice-to-voice translator.
- Collaborative filtering—used to implement recommender systems (“if you liked this movie, you might also like”).
- Text classification—for example, classifying news articles by categories, such as world news, national news, local news, sports, business, entertainment, etc.
- Topic modeling—finding the topics discussed in documents.
- Sarcasm detection—often used with sentiment analysis.
- Text simplification—making text more concise and easier to read.
- Speech to sign language and vice versa—to enable a conversation with a hearing-impaired person.
- Lip reader technology—for people who can't speak, convert lip movement to text or speech to enable conversation.
- Closed captioning—adding text captions to video.

11.9 NATURAL LANGUAGE DATASETS

There's a tremendous number of text data sources available to you for working with natural language processing:

- Wikipedia—some or all of Wikipedia
(<https://meta.wikimedia.org/wiki/Datasets>).
- IMDB (Internet Movie Database)—various movie and TV datasets are available.
- UCIs text datasets—many datasets, including the Spambase dataset.

- Project Gutenberg—50,000+ free e-books that are out-of-copyright in the U.S.
- Jeopardy! dataset—200,000+ questions from the Jeopardy! TV show. A milestone in AI occurred in 2011 when IBM Watson famously beat two of the world’s best Jeopardy! players.
- Natural language processing datasets:
<https://machinelearningmastery.com/datasets-natural-language-processing/>.
- NLTK data: <https://www.nltk.org/data.html>.
- Sentiment labeled sentences data set (from sources including IMDB.com, amazon.com, yelp.com.)
- Registry of Open Data on AWS—a searchable directory of datasets hosted on Amazon Web Services (<https://registry.opendata.aws>).
- Amazon Customer Reviews Dataset—130+ million product reviews (<https://registry.opendata.aws/amazon-reviews/>).
- Pitt.edu corpora (<http://mpqa.cs.pitt.edu/corpora/>).

11.10 WRAP-UP

In this chapter, you performed a broad range of natural language processing (NLP) tasks using several NLP libraries. You saw that NLP is performed on text collections known as corpora. We discussed nuances of meaning that make natural language understanding difficult.

We focused on the TextBlob NLP library, which is built on the NLTK and pattern libraries, but easier to use. You created `TextBlobs` and tokenized them into `Sentences` and `Words`. You determined the part of speech for each word in a `TextBlob`, and you extracted noun phrases.

We demonstrated how to evaluate the positive or negative sentiment of `TextBlobs` and `Sentences` with the TextBlob library’s default sentiment analyzer and with the `NaiveBayesAnalyzer`. You used the TextBlob library’s integration with Google Translate to detect the language of text and perform inter-language translation.

We showed various other NLP tasks, including singularization and pluralization, spell checking and correction, normalization with stemming and lemmatization, and getting word frequencies. You obtained word definitions, synonyms and antonyms from WordNet. You also used NLTK’s stop words list to eliminate stop words from text, and you created n-grams containing groups of consecutive words.

We showed how to visualize word frequencies quantitatively as a bar chart using pandas’ built-in plotting capabilities. Then, we used the `wordcloud` library to visualize word frequencies qualitatively as word clouds. You performed readability assessments using the `Textatistic` library. Finally, you used spaCy to locate named entities and to perform similarity detection among documents. In the next chapter, you’ll continue using natural language processing as we introduce data mining tweets using the Twitter APIs.

12. Data Mining Twitter

Objectives

In this chapter, you'll:

- Understand Twitter's impact on businesses, brands, reputation, sentiment analysis, predictions and more.
- Use Tweepy, one of the most popular Python Twitter API clients for data mining Twitter.
- Use the Twitter Search API to download past tweets that meet your criteria.
- Use the Twitter Streaming API to sample the stream of live tweets as they're happening.
- See that the tweet objects returned by Twitter contain valuable information beyond the tweet text.
- Use the natural language processing techniques from the last chapter to clean and preprocess tweets to prepare them for analysis.
- Perform sentiment analysis on tweets.
- Spot trends with Twitter's Trends API.
- Map tweets using folium and OpenStreetMap.
- Understand various ways to store tweets using techniques discussed throughout this book.

Outline

2.1 Introduction

2.2 Overview of the Twitter APIs

2.3 Creating a Twitter Account

2.4 Getting Twitter Credentials—Creating an App

2.5 What's in a Tweet?

2.6 Tweepy

2.7 Authenticating with Twitter Via Tweepy

2.8 Getting Information About a Twitter Account

2.9 Introduction to Tweepy Cursors: Getting an Account's Followers and Friends

2.9.1 Determining an Account's Followers

2.9.2 Determining Whom an Account Follows

2.9.3 Getting a User's Recent Tweets

2.10 Searching Recent Tweets

2.11 Spotting Trends: Twitter Trends API

2.11.1 Places with Trending Topics

2.11.2 Getting a List of Trending Topics

2.11.3 Create a Word Cloud from Trending Topics

2.12 Cleaning/Preprocessing Tweets for Analysis

2.13 Twitter Streaming API

2.13.1 Creating a Subclass of `StreamListener`

2.13.2 Initiating Stream Processing

2.14 Tweet Sentiment Analysis

2.15 Geocoding and Mapping

2.15.1 Getting and Mapping the Tweets

2.15.2 Utility Functions in `tweetutilities.py`

12.1 INTRODUCTION

We're always trying to predict the future. Will it rain on our upcoming picnic? Will the stock market or individual securities go up or down, and when and by how much? How will people vote in the next election? What's the chance that a new oil exploration venture will strike oil and if so how much would it likely produce? Will a baseball team win more games if it changes its batting philosophy to "swing for the fences?" How much customer traffic does an airline anticipate over the next many months? And hence how should the company buy oil commodity futures to guarantee that it will have the supply it needs and hopefully at a minimal cost? What track is a hurricane likely to take and how powerful will it likely become (category 1, 2, 3, 4 or 5)? That kind of information is crucial to emergency preparedness efforts. Is a financial transaction likely to be fraudulent? Will a mortgage default? Is a disease likely to spread rapidly and, if so, to what geographic area?

Prediction is a challenging and often costly process, but the potential rewards are great. With the technologies in this and the upcoming chapters, we'll see how AI, often in concert with big data, is rapidly improving prediction capabilities.

In this chapter we concentrate on data mining Twitter, looking for the sentiment in tweets. **Data mining** is the process of searching through large collections of data, often big data, to find insights that can be valuable to individuals and organizations. The sentiment that you data mine from tweets could help predict the results of an election, the revenues a new movie is likely to generate and the success of a company's marketing campaign. It could also help companies spot weaknesses in competitors' product offerings.

You'll connect to Twitter via web services. You'll use Twitter's Search API to tap into the enormous base of past tweets. You'll use Twitter's Streaming API to sample the flood of new tweets as they happen. With the Twitter Trends API, you'll see what topics are trending. You'll find that much of what we presented in the "atural Language Processing NLP)" chapter will be useful in building Twitter applications.

As you've seen throughout this book, because of powerful libraries, you'll often perform

significant tasks with just a few lines of code. This is why Python and its robust open-source community are appealing.

Twitter has displaced the major news organizations as the first source for newsworthy events. Most Twitter posts are public and happen in real-time as events unfold globally. People speak frankly about any subject and tweet about their personal and business lives. They comment on the social, entertainment and political scenes and whatever else comes to mind. With their mobile phones, they take and post photos and videos of events as they happen. You'll commonly hear the terms **Twitterverse** and **Twittersphere** to mean the hundreds of millions of users who have anything to do with sending, receiving and analyzing tweets.

What Is Twitter?

Twitter was founded in 2006 as a microblogging company and today is one of the most popular sites on the Internet. Its concept is simple. People write short messages called *tweets*, initially limited to 140 characters but recently increased for most languages to 280 characters. Anyone can generally choose to follow anyone else. This is different from the closed, tight communities on other social media platforms such as Facebook, LinkedIn and many others, where the “following relationships” must be reciprocal.

Twitter Statistics

Twitter has hundreds of millions of users and hundreds of millions of tweets are sent every day with many thousands sent per second.¹ Searching online for “Internet statistics” and “Twitter statistics” will help you put these numbers in perspective. Some “tweeters” have more than 100 million followers. Dedicated tweeters generally post several per day to keep their followers engaged. Tweeters with the largest followings are typically entertainers and politicians. Developers can tap into the live stream of tweets as they're happening. This has been likened to “drinking from a fire hose,” because the tweets come at you so quickly.

¹ <http://www.internetlivestats.com/twitter-statistics/>.

Twitter and Big Data

Twitter has become a favorite big data source for researchers and business people worldwide. Twitter allows regular users free access to a small portion of the more recent tweets. Through special arrangements with Twitter, some third-party businesses (and Twitter itself) offer paid access to much larger portions the all-time tweets database.

Cautions

ou can't always trust everything you read on the Internet, and tweets are no exception. For example, people might use false information to try to manipulate financial markets or influence political elections. Hedge funds often trade securities based in part on the tweet streams they follow, but they're cautious. That's one of the challenges of building *business-critical* or *mission-critical* systems based on social media content.

Going forward, we use web services extensively. Internet connections can be lost, services can change and some services are not available in all countries. This is the real world of cloud-based programming. We cannot program with the same reliability as desktop apps when using web services.

12.2 OVERVIEW OF THE TWITTER APIS

Twitter's APIs are cloud-based web services, so an Internet connection is required to execute the code in this chapter. **Web services** are methods that you call in the cloud, as you'll do with the Twitter APIs in this chapter, the IBM Watson APIs in the next chapter and other APIs you'll use as computing becomes more cloud-based. Each API method has a web service **endpoint**, which is represented by a URL that's used to invoke that method over the Internet.

Twitter's APIs include many categories of functionality, some free and some paid. Most have **rate limits** that restrict the number of times you can use them in 15-minute intervals. In this chapter, you'll use the **Tweepy library** to invoke methods from the following Twitter APIs:

- **Authentication API**—Pass your Twitter credentials (discussed shortly) to Twitter so you can use the other APIs.
- **Accounts and Users API**—Access information about an account.
- **Tweets API**—Search through past tweets, access tweet streams to tap into tweets happening now and more.
- **Trends API**—Find locations of trending topics and get lists of trending topics by location.

See the extensive list of Twitter API categories, subcategories and individual methods at:

<https://developer.twitter.com/en/docs/api-reference-index.html>

Rate Limits: A Word of Caution

Twitter expects developers to use its services responsibly. Each Twitter API method has a **rate limit**, which is the maximum number of requests (that is, calls) you can make during a 15-minute window. Twitter may block you from using its APIs if you continue to call a given API method after that method's rate limit has been reached.

Before using any API method, read its documentation and understand its rate limits.² We'll configure Tweepy to wait when it encounters rate limits. This helps prevent you from exceeding the rate-limit restrictions. Some methods list both user rate limits and app rate limits. All of this chapter's examples use *app rate limits*. User rate limits are for apps that enable individual users to log into Twitter, like third-party apps that interact with Twitter on your behalf, such as smartphone apps from other vendors.

² Keep in mind that Twitter could change these limits in the future.

For details on rate limiting, see

<https://developer.twitter.com/en/docs/basics/rate-limiting>

For specific rate limits on individual API methods, see

<https://developer.twitter.com/en/docs/basics/rate-limits>

and each API method's documentation.

Other Restrictions

Twitter is a goldmine for data mining and they allow you to do a lot with their free services. You'll be amazed at the valuable applications you can build and how these will help you improve your personal and career endeavors. **However, if you do not follow Twitter's rules and regulations, your developer account could be terminated. You should carefully read the following and the documents they link to:**

- Terms of Service: <https://twitter.com/tos>
- Developer Agreement: <https://developer.twitter.com/en/developer-terms/agreement-and-policy.html>
- Developer Policy: <https://developer.twitter.com/en/developer-terms/policy.html>
- Other restrictions: <https://developer.twitter.com/en/developer-terms/more-on-restricted-use-cases>

ou'll see later in this chapter that you can search tweets only for the last seven days and get only a limited number of tweets using the free Twitter APIs. Some books and articles say you can get around those limits by scraping tweets directly from `twitter.com`. However, the Terms of Service explicitly say that “**scraping the Services without the prior consent of Twitter is expressly prohibited.**”

12.3 CREATING A TWITTER ACCOUNT

Twitter requires you to apply for a developer account to be able to use their APIs. Go to <https://developer.twitter.com/en/apply-for-access>

and submit your application. You'll have to register for one as part of this process if you do not already have one. You'll be asked questions about the purpose of your account. You must **carefully** read and agree to Twitter's terms to complete the application, then confirm your email address.

Twitter reviews every application. Approval is not guaranteed. At the time of this writing, *personal-use accounts* were approved immediately. For company accounts, the process was taking from a few days to several weeks, according to the Twitter developer forums.

12.4 GETTING TWITTER CREDENTIALS—CREATING AN APP

Once you have a Twitter developer account, you must obtain **credentials** for interacting with the Twitter APIs. To do so, you'll create an **app**. Each app has separate credentials. To create an app, log into

<https://developer.twitter.com>

and perform the following steps:

1. At the top-right of the page, click the drop-down menu for your account and select **Apps**.
2. Click **Create an app**.
3. In the **App name** field, specify your app's name. If you *send* tweets via the API, this app name will be the tweets' sender. It also will be shown to users if you create applications that require a user to log in via Twitter. We will not do either in this chapter, so a name like "*YourName* Test App" is fine for use with this chapter.

- In the **Application description field**, enter a description for your app. When creating Twitter-based apps that will be used by other people, this would describe what your app does. For this chapter, you can use "Learning to use the Twitter API."
- 5. In the **Website URL** field, enter your website. When creating Twitter-based apps, this is supposed to be the website where you host your app. You can use your Twitter URL: `https://twitter.com/YourUserName`, where *YourUserName* is your Twitter account screen name. For example, `https://twitter.com/nasa` corresponds to the NASA screen name @nasa.
- 6. The **Tell us how this app will be used** field is a description of at least 100 characters that helps Twitter employees understand what your app does. We entered "I am new to Twitter app development and am simply learning how to use the Twitter APIs for educational purposes."
- 7. Leave the remaining fields empty and click **Create**, then carefully review the (lengthy) developer terms and click **Create** again.

Getting Your Credentials

After you complete *Step 7* above, Twitter displays a web page for managing your app. At the top of the page are **App details**, **Keys and tokens** and **Permissions** tabs. Click the **Keys and tokens** tab to view your app's credentials. Initially, the page shows the **Consumer API keys**—the **API key** and the **API secret key**. Click **Create** to get an **access token** and **access token secret**. All four of these will be used to authenticate with Twitter so that you may invoke its APIs.

Storing Your Credentials

As a good practice, do not include your API keys and access tokens (or any other credentials, like usernames and passwords) directly in your source code, as that would expose them to anyone reading the code. You should store your keys in a separate file and never share that file with anyone. ³

³ Good practice would be to use an encryption library such as `bcrypt` (`https://github.com/pyca/bcrypt/`) to encrypt your keys, access tokens or any other credentials you use in your code, then read them in and decrypt them only as you pass them to Twitter.

The code you'll execute in subsequent sections assumes that you place your consumer key, consumer secret, access token and access token secret values into the file `keys.py` shown

below. You can find this file in the `ch12` examples folder:

[lick here to view code image](#)

```
consumer_key='YourConsumerKey'  
consumer_secret='YourConsumerSecret'  
access_token='YourAccessToken'  
access_token_secret='YourAccessTokenSecret'
```

Edit this file, replacing `YourConsumerKey`, `YourConsumerSecret`, `YourAccessToken` and `YourAccessTokenSecret` with your consumer key, consumer secret, access token and access token secret values. Then, save the file.

OAuth 2.0

The consumer key, consumer secret, access token and access token secret are each part of the **OAuth 2.0** authentication process^{4, 5}—sometimes called the *OAuth dance*—that Twitter uses to enable access to its APIs. The Tweepy library enables you to provide the consumer key, consumer secret, access token and access token secret and handles the OAuth 2.0 authentication details for you.

4

<https://developer.twitter.com/en/docs/basics/authentication/overview>.

5 <https://oauth.net/>.

12.5 WHAT'S IN A TWEET?

The Twitter API methods return JSON objects. **JSON (JavaScript Object Notation)** is a text-based data-interchange format used to represent objects as collections of name–value pairs. It's commonly used when invoking web services. JSON is both a human-readable and computer-readable format that makes data easy to send and receive across the Internet.

JSON objects are similar to Python dictionaries. Each JSON object contains a list of *property names* and *values*, in the following curly braced format:

```
{propertyName1: value1, propertyName2: value2}
```

As in Python, JSON lists are comma-separated values in square brackets:

```
[value1, value2, value3]
```